Ecoole Supérieur de Technologie de Khénifra
Département : Génie de l'informatique et mathématiques
Filière : ingénierie de données et intelligence artificielle
Module : Apprentissage automatique

TP 2 : KNN

Les TP sont notés. Les étudiants doivent commenter et sauvegarder le code de chaque exercice.

Exercice 1 :

Objectif :

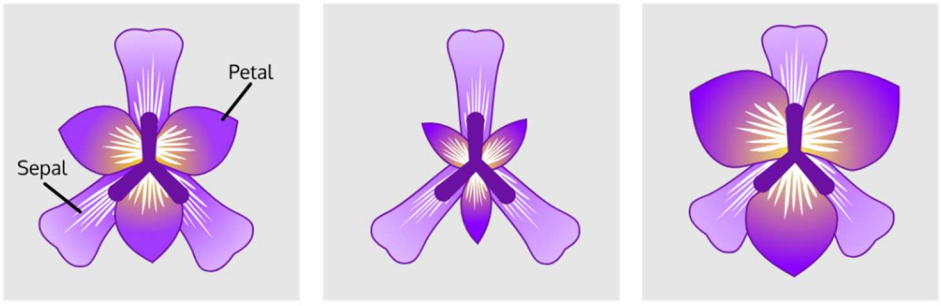
- Comprendre le principe de la classification par **plus proches voisins**.
- Appliquer KNN sur un dataset réel.
- Évaluer la performance du modèle.

Iris dataset (Fisher's Iris Data Set)

- Disponible directement via **scikit-learn** ou sur UCI Machine Learning Repository : https://archive.ics.uci.edu/ml/datasets/iris

Description des variables :

- SepalLengthCm : longueur du sépale
- SepalWidthCm : largeur du sépale
- PetalLengthCm : longueur du pétale
- PetalWidthCm : largeur du pétale
- Species : classe (Setosa, Versicolor, Virginica)



Iris Versicolor | Iris Setosa | Iris Virginica

Travail demandé
1.Chargement et exploration du dataset

- Charger le dataset Iris depuis scikit-learn.

- Afficher la taille, les 5 premières lignes, et la distribution des classes.

2. Visualisation

- Tracer un nuage de points (scatter plot) PetalLength vs PetalWidth, colorié par espèce.
- Commenter la séparabilité des classes.

3. Implémentation de KNN

- Diviser le dataset en **70% entraînement / 30% test**.
- Appliquer **KNN** avec k=3.
- Prédire les classes des individus du test.

4. Évaluation

- Construire la **matrice de confusion**.
- Calculer la **précision (accuracy)**.
- Expérimenter avec différentes valeurs de k (1, 3, 5, 7, 9).
- Tracer une courbe de la précision en fonction de k.

5. Discussion

- Quel est le meilleur k pour ce dataset ?
- Que se passe-t-il si k est trop petit ou trop grand ?

Exercice 2 :

Objectif :

- Appliquer KNN sur un **dataset concret du monde réel**.
- Comparer la performance du modèle selon différentes configurations.

Dataset proposé :

Student Performance Dataset (Kaggle / UCI)
Lien : https://archive.ics.uci.edu/ml/datasets/student+performance

Description des variables (extrait) :

- studytime : nombre d'heures d'étude par semaine.
- failures : nombre d'échecs antérieurs.
- absences : nombre d'absences.
- G1, G2, G3 : notes aux trois trimestres (G3 est la note finale).

Variable cible :

- Transformer G3 en une variable binaire :
    - **Pass** (si G3 ≥ 10)
    - **Fail** (si G3 < 10)

Travail demandé

1. Préparation du dataset

- Télécharger et charger le dataset (CSV).
- Créer une nouvelle colonne result = *Pass* ou *Fail*.
- Sélectionner des variables explicatives : studytime, failures, absences, G1, G2.

2. Analyse exploratoire

- Visualiser la distribution des notes (G3).
- Vérifier l'équilibre des classes (nombre de *Pass* vs *Fail*).

3. Implémentation KNN

- Diviser les données en **70% entraînement / 30% test**.
- Appliquer **KNN** avec k=5.
- Prédire le résultat des étudiants.

4. Évaluation

- Afficher la matrice de confusion.
- Calculer les métriques : **accuracy, recall, precision, F1-score**.
- Comparer avec d'autres valeurs de k (1, 3, 7, 11).

5. Discussion

- Quelles variables semblent les plus discriminantes ?
- Comment la variation de k influence-t-elle les résultats ?
- Quels sont les avantages et limites de KNN pour ce type de données éducatives ?

Exercice 3:

Objectif: Créer un classifieur KNN pour prédire si une personne aura un diagnostic de diabète positif ou non.

Nous avons un ensemble de données de 768 personnes qui ont été diagnostiquées positives avec le diabète et d'autres ont été diagnostiquées négatives. Suivre les étapes suivantes pour créer le classifieur KNN et l'utiliser pour faire des tests :

1.) Importer les bibliothèques nécessaires : pandas, numpy, matplotlib, et sklearn. La bibliothèque sklearn comprend plusieurs fonctions :

- from sklearn.model selection import train test split
- from sklearn.preprocessing import StandardScaler
- from sklearn.neighbors import KNeighborsClassifier
- from sklearn.metrics import confusion matrix
- from sklearn.metrics import ConfusionMatrixDisplay
- from sklearn.metrics import f1 score
- from sklearn.metrics import accuracy score

2.) Télécharger la base de données 'diabetes.csv'. Utiliser le lien suivant https://www.kaggle.com/ datasets/saurabh00007/diabetescsv

3.) Lire la base de données avec pandas.

4.) Afficher la longueur (nombre d'enregistrements) de la base de données.

5.) Afficher les 10 premières lignes de la base de données.

6.) On remarque que les colonnes Glucose', BloodPressure', SkinThickness', Insulin', et BMI' contiennent des valeurs nulles (0). Remplacer les valeurs 0 par la valeur moyenne de la colonne correspondante.

7.) Afficher les 10 premières lignes de la base de données après avoir enlevé les valeurs nulles.

8.) Préciser les entrées X du classifieur KNN (les colonnes Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, et Age) et la sortie y du classifieur (la colonne Outcome)

9.) Diviser la base de données en 2 parties: apprentissage (80% du nombre total des enregistrements) et test (20%). Utiliser la fonction train test split.

10.) Afficher les longueurs des données de la base d'apprentissage et celles de test.

11.) Mettre les données d'entrée de la base d'apprentissage et celles de test `a l'échelle (Features scaling). Utiliser les fonctions fit_transform() pour les entrées de la base d'apprentissage et les entrées de la base de test.

12.) Afficher les entrées de la base d'apprentissage et de test après mise `a l'échelle.

13.) Définir le classifieur KNN. Utiliser la fonction KNeighborsClassifier(). Utiliser la distance euclidienne pour calculer la similarité entre les données. Faire des tests avec d'autres distances.

14.) Faire l'entrainement du modèle sur les données de la base d'apprentissage en utilisant le classifieur KNN et la fonction fit().

15.) Calculer les prédictions sur les données de la base de test en utilisant le classifieur KNN et la fonction predict().

16.) Faire l'évaluation du modèle avec la matrice de confusion, F1-score, et la précision (accuracy).

17.) Afficher la matrice de confusion.